Full Stack Development with MERN

1.INTRODUCTION

1.1Project Title:

Booknest : Where Stories Nestle

1.2Team Members:

- Kurba Yashashwani
- Anitha Alluri
- Gurappagari Chandusree
- Jayanthi Bapandoddi

2 Project Overview: Book Store Application

2.1 Purpose :

The Book Store Application is a full-stack web platform designed to provide book enthusiasts with a seamless online shopping experience. Built using the MERN stack (MongoDB, Express.js, React, Node.js), its primary goal is to connect readers with a vast collection of books, enabling them to discover, explore, and purchase titles effortlessly from any device. The application aims to replicate the joy of browsing a physical bookstore while offering the convenience of modern e-commerce, tailored to individual reading preferences.

Goals :

· **Accessibility:** Allow users to browse and buy books anytime, anywhere, with a responsive design that works on desktops, tablets, and smartphones.

· **User-Centric Experience:** Provide intuitive navigation, personalized recommendations, and detailed book information to help users make informed choices.

· **Secure & Reliable:** Implement robust authentication and authorization to protect user data and ensure secure transactions.

· **Scalable Inventory:** Enable easy management of book listings, stock levels, and orders through an admin interface.

· **Community Engagement:** Encourage user interaction through reviews and ratings, fostering a community of book lovers.

2.2 Key Features :

1. User Authentication & Profiles

· Secure registration and login using JWT (JSON Web Tokens).

· Password hashing with bcrypt for data protection.

· User profiles with editable personal information, shipping addresses, and order history.

2. Book Catalog & Discovery

· Comprehensive book listings with details such as title, author, genre, description, price, and availability.

· Advanced search and filtering by genre, author, price range, and ratings.

· Sorting options (newest, price low to high, highest rated, etc.).

· Featured books and new releases sections on the homepage

3. Shopping Cart

· Add/remove books, adjust quantities, and view real-time cart totals.

· Persistent cart using local storage (or database for logged-in users).

· Coupon code support for discounts.

4. Checkout & Order Processing

· Streamlined checkout with shipping address and payment method selection.

· Support for multiple payment options (credit card, PayPal, cash on delivery).

· Order summary and confirmation with estimated delivery dates.

· Automatic inventory update upon order placement.

5. Order Management

· Order history page for users to track current and past orders.

· Real-time order status updates (Processing, Confirmed, Shipped, Delivered, Cancelled).

· Detailed order view with items, totals, and shipping information.

6. Reviews & Ratings

· Users can rate and review books they've purchased.

· Average rating display on book cards and detail pages.

· Helps other users make informed decisions.

7. Admin Dashboard

· Admin role with privileges to add, edit, and delete books.

· Manage inventory, view all orders, and update order statuses.

· User management (view, suspend, or promote users).

8. Responsive & Modern UI

· Built with React and Bootstrap for a clean, mobile-first interface.

· Smooth animations and interactive elements enhance user engagement.

· Accessible design following best practices.

9. Security & Data Protection

· Input validation and sanitization on both client and server.

· Protected API routes with middleware.

· Environment variables for sensitive configuration.

This application delivers a complete e-commerce solution for book retailers, combining powerful backend functionality with an engaging frontend to create a delightful shopping experience for e

very reader.

3 Architecture

3.1 Frontend Architecture (React) :

The frontend is built with React, utilizing a component-based architecture for modularity and reusability.

Key Aspects :

- **Component Hierarchy:**

  · Root App component sets up routing and global providers.

  · Layout components (Header, Footer) are shared across pages.

  · Page components (HomePage, BooksPage, BookDetailsPage, CartPage, CheckoutPage, LoginPage, RegisterPage, ProfilePage, OrdersPage) correspond to routes.

  · Reusable UI components (BookCard, Rating, PrivateRoute, Spinner, Modal) enhance consistency and reduce duplication.

- **State Management:**

  · React Context API manages global state:

  · AuthContext handles user authentication (user info, token, login/logout).

  · CartContext manages shopping cart (items, quantity updates, totals).

  · Local state with useState/useReducer for component-specific data (e.g., form inputs, filters).

- **Routing:**

  · React Router v6 enables client-side navigation.

  · Protected routes are implemented via a PrivateRoute wrapper that checks authentication.

- **API Integration:**

  · Axios is configured with base URL, request interceptors (to attach JWT tokens), and response interceptors (for error handling).

  · API calls are organized in service files or custom hooks (e.g., useBooks, useAuth).

- **Styling:**

  · Bootstrap 5 and React-Bootstrap provide responsive, pre-styled components.

  · Custom CSS and React Icons (Font Awesome) enhance the user interface.

- **Performance Optimizations:**

  · Lazy loading of routes with React.lazy and Suspense for code splitting.

  · Memoization (React.memo, useMemo, useCallback) prevents unnecessary re-renders.

3.2 Backend Architecture (Node.js & Express.js) :

The backend exposes a RESTful API built with Node.js and Express, following a modular MVC-like structure.

Project Structure :

backend/

```
├── models/        # Mongoose data models (User, Book, Order)
├── controllers/   # Business logic for each endpoint
├── routes/        # API route definitions
├── middleware/    # Custom middleware (auth, error handling)
├── config/        # Database connection and environment config
├── utils/         # Helper functions
├── server.js      # Entry point
└── .env           # Environment variables
```

Key Layers :

- **Routes:**
  - · Resource-based routing (authRoutes, bookRoutes, orderRoutes).
  - · Each route file maps HTTP methods to corresponding controller functions.
- **Controllers:**
  - · Encapsulate business logic:
  - · authController: Registration, login, profile management, JWT issuance.
  - · bookController: CRUD for books, search, filtering, reviews.
  - · orderController: Order creation, retrieval, status updates (admin).
  - · Controllers interact with Mongoose models and return JSON responses.
- **Middleware:**
  - · Authentication: Verifies JWT from Authorization header and attaches user to req.
  - · Admin: Checks if authenticated user has admin role.
  - · Validation: Uses express-validator to validate request data.
  - · Error Handling: Centralized error handler formats and returns error responses.

➢ **Security:**

· Passwords hashed with bcryptjs.

· JWT for stateless authentication.

· Environment variables protect sensitive data (database URI, JWT secret).

· CORS enabled to allow frontend access.

· Input sanitization prevents injection attacks.

· API Endpoints (examples):

· POST /api/auth/register – register new user

· POST /api/auth/login – login, receive token

· GET /api/books – list books (with filtering & pagination)

· GET /api/books/:id – get single book details

· POST /api/orders – create order (protected)

· GET /api/orders/my-orders – get user's orders (protected)

3.3 Database Architecture (MongoDB) :

The database is MongoDB, with Mongoose as the ODM for schema definition and data interaction.

Collections & Schemas :

User Collection

javascript

```
{
  _id: ObjectId,
  name: String (required),
  email: String (unique, required),
  password: String (hashed, required),
  phone: String,
  address: {
    street: String, city: String, state: String, zipCode: String, country: String
  },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  createdAt: Date
}
```

Book Collection

javascript

```javascript
{
  _id: ObjectId,
  title: String (required),
  author: String (required),
  isbn: String (unique),
  genre: String (enum: ['Fiction', 'Non-Fiction', ...]),
  description: String,
  price: Number (min 0),
  stockQuantity: Number (min 0),
  publisher: String,
  publicationYear: Number,
  pages: Number,
  language: String,
  coverImage: String (URL),
  rating: { type: Number, default: 0, min: 0, max: 5 },
  reviews: [{
    user: { type: ObjectId, ref: 'User' },
    rating: Number,
    comment: String,
    date: { type: Date, default: Date.now }
  }],
  createdAt: Date
}
```

Order Collection

javascript

```
{
  _id: ObjectId,
  user: { type: ObjectId, ref: 'User', required: true },
  items: [{
    book: { type: ObjectId, ref: 'Book', required: true },
    quantity: Number,
    price: Number  // snapshot of book price at order time
  }],
  totalAmount: Number,
  shippingAddress: { street, city, state, zipCode, country },
  paymentMethod: String (enum: 'Credit Card', 'PayPal', 'Cash on Delivery'),
  paymentStatus: { type: String, enum: ['Pending', 'Completed', 'Failed'], default: 'Pending' },
  orderStatus: { type: String, enum: ['Processing', 'Confirmed', 'Shipped', 'Delivered', 'Cancelled'], default: 'Processing' },
  trackingNumber: String,
  estimatedDelivery: Date,
  createdAt: Date
}
```

Relationships & Indexing

· User → Orders: One-to-many via user reference in Order.

· Book → Reviews: Embedded sub-documents (reviews are tightly coupled with the book).

· Order → Books: Many-to-many via items array with book references and price snapshots.

Indexes:

· Text indexes on title, author, description for search.

· Indexes on genre, author, price for efficient filtering.

· Index on createdAt for sorting by newest.

Data Interactions

· Mongoose provides schema validation, middleware (e.g., pre-save hooks for password hashing), and population for referenced documents.

· Queries use Mongoose’s fluent API:

· User.find({ email }) for authentication.

· Book.find(filter).sort(sort).skip(skip).limit(limit) for paginated listings.

· Order.find({ user: userId }).populate('items.book') for retrieving user orders with book details.

· Atomic updates (e.g., $inc for stock adjustment) ensure consistency when orders are placed or cancelled.

Data Consistency

· Inventory is updated transaction-like: on order creation, book stock is decremented; on cancellation, stock is incremented.

· Price snapshots in orders preserve historical accuracy even if book prices change later.

This three-tier architecture ensures a clear separation of concerns, scalability, and maintainability, enabling smooth development and future enhancements.

4 Setup Instructions

4.1 Prerequisites:

Before you begin, ensure you have the following installed on your system:

· Node.js (v14 or higher)

· MongoDB

· Git (optional)

· A code editor (e.g., VS Code)

4.2 Installation :

1. Clone the Repository

Open a terminal and run:

bash

```
git clone <repository-url>
cd book-store-app
```

If you don't have Git, you can download the source code as a ZIP and extract it.

2. Backend Setup

Navigate to the backend folder:

bash

```
cd backend
```

Install backend dependencies:

bash

```
npm install
```

Configure environment variables:

Create a .env file in the backend folder and add the following:

env

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/bookstore
JWT_SECRET=your-super-secret-key-change-this
NODE_ENV=development
```

Explanation of environment variables:

· **PORT:** The port on which the backend server will run (default 5000).

· **MONGODB_URI:** Your MongoDB connection string. For local MongoDB, use mongodb://localhost:27017/bookstore. For MongoDB Atlas, use the connection string provided by Atlas (e.g., mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/bookstore?retryWrites=true&w=majority).

· **JWT_SECRET:** A strong random string used to sign JSON Web Tokens (e.g., a long, complex passphrase).

· **NODE_ENV:** Set to development for local development; use production in production.

3. Frontend Setup

Open a new terminal window (keep the backend running) and navigate to the frontend folder:

bash

cd frontend

Install frontend dependencies

bash

npm install

Configure the API connection (if needed):

The frontend is configured to proxy API requests to the backend during development. In frontend/package.json, ensure the proxy field points to your backend URL:

json

"proxy": "http://localhost:5000"

If your backend runs on a different port, update it accordingly.

Start the React development server:

bash

npm start

The app will automatically open in your browser at http://localhost:3000**.**

4. Verify the Setup

· Navigate to http://localhost:3000 – you should see the home page.

· Try logging in with the seeded user credentials:

· User: john@example.com / user123

· Admin: admin@bookstore.com / admin123

· Browse books, add items to the cart, and proceed to checkout.

Troubleshooting

· MongoDB connection error: Ensure MongoDB is running locally (mongod) or your Atlas connection string is correct and your IP is whitelisted.

· Port already in use: Change the PORT in .env and update the frontend proxy accordingly.

· Dependency errors: Run npm install again, or delete node_modules and package-lock.json and reinstall.

· CORS issues: The backend includes the cors middleware; if you still encounter CORS errors, check that the frontend origin is allowed.

Running in Production

For production deployment, you'll need to:

1. Build the frontend: npm run build (in the frontend folder).

2. Serve the static files with a web server (e.g., Nginx) or use a service like Netlify/Vercel.

3. Deploy the backend to a cloud platform (e.g., Heroku, DigitalOcean, AWS) and set the environment variables there.

4. Update the frontend API base URL to point to your deployed backend (remove the proxy and set REACT_APP_API_URL in a .env file).

5 Folder Structure

5.1 Client – React Frontend Structure

The frontend is built with React and follows a modular, component-based architecture to ensure scalability, reusability, and maintainability. The key aspects of the client structure are:

1. Folder Organization

```
frontend/
├── public/              # Static assets (index.html, favicon, etc.)
├── src/
│   ├── components/      # Reusable UI components
│   │   ├── Header.js
│   │   ├── Footer.js
│   │   ├── BookCard.js
│   │   ├── PrivateRoute.js
│   │   ├── Spinner.js
│   │   └── ...
│   ├── pages/           # Page-level components (each corresponds to a route)
│   │   ├── HomePage.js
│   │   ├── BooksPage.js
│   │   ├── BookDetailsPage.js
│   │   ├── CartPage.js
│   │   ├── CheckoutPage.js
│   │   ├── LoginPage.js
│   │   ├── RegisterPage.js
│   │   ├── ProfilePage.js
│   │   └── OrdersPage.js
│   ├── context/         # React Context providers for global state
│   │   ├── AuthContext.js   # Manages user authentication state
│   │   └── CartContext.js   # Manages shopping cart state
│   ├── services/        # API integration layer
```

```
│   │   └── api.js          # Axios instance with interceptors
│   ├── App.js              # Root component with routing and providers
│   ├── index.js            # Entry point
│   └── index.css           # Global styles
├── package.json
└── README.md
```

2. Component Architecture

· Presentational Components: Reusable, stateless components (e.g., BookCard, Rating, Spinner) that receive data via props and focus on UI rendering.

· Container Components: Page-level components that handle data fetching, state, and business logic (e.g., BooksPage fetches books and manages filters).

· Layout Components: Header and Footer are present on all pages and are composed in App.js.

3. State Management

· React Context API is used for global state:

  · AuthContext: Stores user info, authentication token, and provides login/logout functions.

  · CartContext: Manages cart items, quantities, and total calculation; persists cart in localStorage for guest users.

· Local state is managed with useState and useReducer for component-specific data (e.g., form inputs, filter selections).

4. Routing

· React Router v6 handles client-side navigation.

· Routes are defined in App.js using <Routes> and <Route>.

· Protected routes are implemented with a PrivateRoute wrapper that checks authentication status and redirects to login if needed.

5. API Integration

· Axios is configured in services/api.js with a base URL pointing to the backend.

· Request interceptors automatically attach the JWT token to the Authorization header for authenticated requests.

· Response interceptors handle common errors (e.g., 401 Unauthorized) and display toast notifications.

· Data fetching is performed inside useEffect hooks or custom hooks (e.g., useBooks, useOrders).

6. Styling

· Bootstrap 5 and React-Bootstrap provide responsive, pre-built components.

· Custom CSS is added via index.css and component-specific inline styles where needed.

· React Icons (Font Awesome) are used for icons.

7. Performance Optimizations

· Code splitting using React.lazy and Suspense for route-based chunk loading.

· Memoization with React.memo, useMemo, and useCallback to prevent unnecessary re-renders.

5.2 Server – Node.js Backend Organization

The backend is a RESTful API built with Node.js and Express.js, following a clean MVC-like architecture. The code is organised for clarity, maintainability, and scalability.

1. Folder Structure

```
backend/
├── models/          # Mongoose data models (schemas)
│   ├── User.js
│   ├── Book.js
│   └── Order.js
├── controllers/     # Business logic for each resource
│   ├── authController.js
│   ├── bookController.js
│   └── orderController.js
├── routes/          # API route definitions
│   ├── authRoutes.js
│   ├── bookRoutes.js
│   └── orderRoutes.js
├── middleware/      # Custom middleware functions
│   └── authMiddleware.js   # Authentication & authorisation
├── config/          # Configuration files
│   └── db.js        # MongoDB connection
```

```
├── utils/          # Helper functions (optional)
├── server.js       # Entry point – initialises the app
├── seed.js         # Database seeding script
├── .env            # Environment variables
└── package.json
```

2. Application Flow

· Routes define the API endpoints and map them to controller functions.

· Controllers handle request validation, interact with models, and send responses.

· Models define the database schema and provide an abstraction layer over MongoDB.

· Middleware processes requests before they reach controllers (e.g., authentication, error handling).

3. Key Components

Routes

· Each resource has its own route file (authRoutes.js, bookRoutes.js, orderRoutes.js).

· Routes are prefixed (e.g., /api/auth) and mounted in server.js.

· HTTP methods (GET, POST, PUT, DELETE) are mapped to controller methods.

Controllers

· Contain the core business logic.

· Use Mongoose methods to query the database.

· Return JSON responses with appropriate HTTP status codes.

· Example: bookController.js exports functions like getAllBooks, getBookById, createBook, etc.

Models

· Define Mongoose schemas with validation, default values, and data types.

· Include middleware (e.g., pre-save hooks for password hashing).

· Establish relationships via ObjectId references and population.

Middleware

· Authentication Middleware: Verifies JWT from the Authorization header and attaches the user object to req.

· Admin Middleware: Checks if the authenticated user has an admin role.

· Validation Middleware: Uses express-validator to sanitise and validate incoming data.

· Error Handling Middleware: Catches errors and sends a consistent error response.

4. Security Measures

· Passwords are hashed with bcryptjs before storage.

· JWT tokens are used for stateless authentication.

· Environment variables (.env) store sensitive data like database URI and JWT secret.

· CORS is enabled to allow requests from the frontend.

· Input validation and sanitisation prevent injection attacks.

5. API Endpoint Summary

Method Endpoint Description Access

POST /api/auth/register Register a new user Public

POST /api/auth/login Login and receive JWT Public

GET /api/auth/profile Get current user profile Private

PUT /api/auth/profile Update user profile Private

GET /api/books Get all books (filtering, pagination) Public

GET /api/books/:id Get single book by ID Public

POST /api/books Create a new book Admin only

PUT /api/books/:id Update a book Admin only

DELETE /api/books/:id Delete a book Admin only

POST /api/books/:id/reviews Add a review to a book Private

POST /api/orders Create a new order Private

GET /api/orders/my-orders Get logged-in user's orders Private

GET /api/orders/:id Get order details Private/Admin

PUT /api/orders/:id/cancel Cancel an order Private

6. Database Connection

· MongoDB connection is established in config/db.js using Mongoose.

· The connection URI is read from the environment variable MONGODB_URI.

· Connection events are logged for monitoring.

7. Error Handling

· Centralised error handler catches both synchronous and asynchronous errors.

· Returns a JSON response with an appropriate HTTP status code and error message.

· In development, stack traces are included; in production, they are omitted.

This structured approach ensures that the backend remains maintainable, testable, and ready for future enhancements.

6 Running the Application

6.1 Start the Backend Server

Open a terminal and navigate to the backend directory:

bash

cd backend

npm start

The server will start at http://localhost:5000.

(Use npm run dev instead if you want automatic restarts on code changes – requires nodemon.)

6.2 Start the Frontend Server

Open another terminal and navigate to the frontend directory:

bash

cd frontend

npm start

The React development server will start at http://localhost:3000 and automatically open the app in your browser.

7 API Documentation

Base URL

All endpoints are relative to http://localhost:5000/api (when running locally).

Protected endpoints require a JWT token in the Authorization header:

Authorization: Bearer <your_token>

Authentication Endpoints

1. Register a New User

· URL: /auth/register

· Method: POST

· Description: Creates a new user account.

· Request Body (JSON):

```json
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "secret123",
  "phone": "123-456-7890",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "state": "NY",
    "zipCode": "10001",
    "country": "USA"
  }
}
```

· Success Response (201 Created):

```json
```

```
{
  "_id": "60d21b4667d0d8992e610c85",
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "123-456-7890",
  "role": "user",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

· Error Response (400 Bad Request):

```json
{
  "message": "User already exists"
}
```

2. Login

· URL: /auth/login

· Method: POST

· Description: Authenticates a user and returns a JWT token.

· Request Body (JSON):

```json
{
  "email": "john@example.com",
  "password": "secret123"
}
```

· Success Response (200 OK):

```json
{
  "_id": "60d21b4667d0d8992e610c85",
```

```
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "123-456-7890",
  "role": "user",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

· Error Response (401 Unauthorized):

```json
{
  "message": "Invalid email or password"
}
```

3. Get Current User Profile

· URL: /auth/profile

· Method: GET

· Description: Returns the profile of the currently authenticated user.

· Headers: Authorization: Bearer <token>

· Success Response (200 OK):

```json
{
  "_id": "60d21b4667d0d8992e610c85",
  "name": "John Doe",
  "email": "john@example.com",
  "phone": "123-456-7890",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "state": "NY",
    "zipCode": "10001",
```

```
    "country": "USA"
  },
  "role": "user",
  "createdAt": "2024-01-15T10:30:00.000Z"
}
```

· Error Response (401 Unauthorized):

```json
{
  "message": "Not authorized, no token"
}
```

4. Update User Profile

· URL: /auth/profile

· Method: PUT

· Description: Updates the authenticated user's profile.

· Headers: Authorization: Bearer <token>

· Request Body (JSON) – all fields optional:

```json
{
  "name": "Jane Doe",
  "phone": "987-654-3210",
  "address": {
    "street": "456 Oak Ave",
    "city": "Los Angeles",
    "state": "CA",
    "zipCode": "90001",
    "country": "USA"
  },
  "password": "newpassword123"
```

```
}
```

· Success Response (200 OK):

```json
{
  "_id": "60d21b4667d0d8992e610c85",
  "name": "Jane Doe",
  "email": "john@example.com",
  "phone": "987-654-3210",
  "address": {
    "street": "456 Oak Ave",
    "city": "Los Angeles",
    "state": "CA",
    "zipCode": "90001",
    "country": "USA"
  },
  "role": "user"
}
```

Book Endpoints

1. Get All Books

· URL: /books

· Method: GET

· Description: Retrieves a list of books with optional filtering, sorting, and pagination.

· Query Parameters (all optional):

  · page: Page number (default: 1)

  · limit: Number of items per page (default: 12)

  · genre: Filter by genre (e.g., Fiction)

  · author: Filter by author (partial match)

  · minPrice: Minimum price

  · maxPrice: Maximum price
  · sortBy: Sort field and order, e.g., price:asc, rating:desc, createdAt:desc
· Success Response (200 OK):
  ```json
  {
    "books": [
      {
        "_id": "60d21b4667d0d8992e610c86",
        "title": "The Great Gatsby",
        "author": "F. Scott Fitzgerald",
        "genre": "Fiction",
        "price": 12.99,
        "rating": 4.5,
        "coverImage": "https://...",
        "stockQuantity": 50
      },
    ],
    "currentPage": 1,
    "totalPages": 10,
    "totalBooks": 120
  }
**2. Get Single Book by ID**
· URL: /books/:id
· Method: GET
· Description: Returns detailed information about a specific book, including reviews.
· Path Parameter: id – Book ID
· Success Response (200 OK):
  ```json
  {
    "_id": "60d21b4667d0d8992e610c86",

"title": "The Great Gatsby",
"author": "F. Scott Fitzgerald",
"isbn": "978-0-7432-7356-5",
"genre": "Fiction",
"description": "The story of the mysteriously wealthy Jay Gatsby...",
"price": 12.99,
"stockQuantity": 50,
"publisher": "Scribner",
"publicationYear": 1925,
"pages": 180,
"language": "English",
"coverImage": "https://...",
"rating": 4.5,
"reviews": [
{
"user": {
"_id": "60d21b4667d0d8992e610c85",
"name": "John Doe"
},
"rating": 5,
"comment": "Amazing book!",
"date": "2024-02-01T12:00:00.000Z"
}
],
"createdAt": "2024-01-01T10:00:00.000Z"
}

· Error Response (404 Not Found):

```json
{
"message": "Book not found"
```

}

3. Create a New Book (Admin only)

· URL: /books

· Method: POST

· Description: Adds a new book to the inventory. Requires admin privileges.

· Headers: Authorization: Bearer <admin_token>

· Request Body (JSON):

```json
{
  "title": "New Book",
  "author": "Author Name",
  "isbn": "978-1-2345-6789-0",
  "genre": "Fiction",
  "description": "A compelling story...",
  "price": 19.99,
  "stockQuantity": 100,
  "publisher": "Publisher Name",
  "publicationYear": 2024,
  "pages": 320,
  "language": "English",
  "coverImage": "https://example.com/cover.jpg"
}
```

· Success Response (201 Created): Returns the created book object.

· Error Response (403 Forbidden):

```json
{
  "message": "Not authorized as admin"
}
```

4. Update a Book (Admin only)

· URL: /books/:id

· Method: PUT

· Description: Updates an existing book. Requires admin privileges.

· Headers: Authorization: Bearer <admin_token>

· Path Parameter: id – Book ID

· Request Body: Any fields to update (same as create).

· Success Response (200 OK): Returns the updated book.

5. Delete a Book (Admin only)

· URL: /books/:id

· Method: DELETE

· Description: Removes a book from the inventory. Requires admin privileges.

· Headers: Authorization: Bearer <admin_token>

· Success Response (200 OK):

```json
{
  "message": "Book removed"
}
```

6. Add a Review to a Book

· URL: /books/:id/reviews

· Method: POST

· Description: Adds a review and rating for a book. User must be authenticated.

· Headers: Authorization: Bearer <token>

· Path Parameter: id – Book ID

· Request Body:

```json
{
  "rating": 5,
  "comment": "Excellent read!"
}
```

· Success Response (201 Created):

```json
```

```
{
  "message": "Review added"
}
```

· Error Response (400 Bad Request):

```json
{
  "message": "Book already reviewed"
}
```

7. Get Featured Books

· URL: /books/featured

· Method: GET

· Description: Returns a list of books with rating ≥ 4 (featured).

· Success Response (200 OK): Array of book objects.

8. Get All Genres

· URL: /books/genres

· Method: GET

· Description: Returns a list of distinct book genres.

· Success Response (200 OK):

```json
["Fiction", "Non-Fiction", "Mystery", "Science Fiction", ...]
```

9. Search Books

· URL: /books/search

· Method: GET

· Description: Searches books by title, author, or description.

· Query Parameters:

  · q: Search query (required)

  · genre: Optional genre filter

  · author: Optional author filter

· Success Response (200 OK): Array of matching books.

Order Endpoints

1. Create a New Order

· URL: /orders

· Method: POST

· Description: Places a new order. User must be authenticated.

· Headers: Authorization: Bearer <token>

· Request Body (JSON):

```json
{
  "items": [
    {
      "book": "60d21b4667d0d8992e610c86",
      "quantity": 2
    },
    {
      "book": "60d21b4667d0d8992e610c87",
      "quantity": 1
    }
  ],
  "shippingAddress": {
    "street": "123 Main St",
    "city": "New York",
    "state": "NY",
    "zipCode": "10001",
    "country": "USA"
  },
  "paymentMethod": "Credit Card"
}
```

· Success Response (201 Created):

```json
{
  "_id": "60d21b4667d0d8992e610c90",
  "user": "60d21b4667d0d8992e610c85",
  "items": [
    {
      "book": {
        "_id": "60d21b4667d0d8992e610c86",
        "title": "The Great Gatsby"
      },
      "quantity": 2,
      "price": 12.99
    }
  ```
  ],
  "totalAmount": 38.97,
  "shippingAddress": { ... },
  "paymentMethod": "Credit Card",
  "paymentStatus": "Pending",
  "orderStatus": "Processing",
  "createdAt": "2024-03-01T10:30:00.000Z"
}
```

**2. Get Current User's Orders**

· URL: /orders/my-orders

· Method: GET

· Description: Returns all orders placed by the authenticated user.

· Headers: Authorization: Bearer <token>

· Success Response (200 OK): Array of order objects (populated with book details).

**3. Get Order by ID**

· URL: /orders/:id
```

· Method: GET

· Description: Returns details of a specific order. Users can only access their own orders; admins can access any order.

· Headers: Authorization: Bearer <token>

· Path Parameter: id – Order ID

· Success Response (200 OK): Full order object with populated user and book details.

· Error Response (403 Forbidden):

```json
{
  "message": "Not authorized"
}
```

4. Cancel an Order

· URL: /orders/:id/cancel

· Method: PUT

· Description: Cancels an order if its status is still "Processing" or "Confirmed". Inventory is restored.

· Headers: Authorization: Bearer <token>

· Path Parameter: id – Order ID

· Success Response (200 OK):

```json
{
  "message": "Order cancelled successfully"
}
```

· Error Response (400 Bad Request):

```json
{
  "message": "Order cannot be cancelled at this stage"
}
```

Error Responses

All endpoints may return the following error structure in case of server errors (500):

```json
{
  "message": "Server error"
}
```

For validation errors (400), an array of errors may be returned:

```json
{
  "errors": [
    { "msg": "Password must be at least 6 characters", "param": "password" }
  ]
}
```

8 Authentication

Authentication & Authorisation

The application uses JSON Web Tokens (JWT) for stateless authentication and role-based authorisation. This approach ensures secure, scalable user management without maintaining server-side sessions.

1. Authentication Process

· Registration

When a user registers via POST /api/auth/register, their password is hashed using bcryptjs before being stored in the database. The response includes a JWT token (along with user details) so the user is automatically logged in after registration.

· Login

At POST /api/auth/login, the provided email and password are validated. If credentials are correct, a JWT token is generated using a secret key stored in the environment variables (JWT_SECRET). The token is signed with the user's ID and an expiration time (e.g., 30 days). The token is then returned to the client.

· Token Storage & Usage

The client (frontend) stores the token in localStorage (or in memory, but typically localStorage for persistence). For every subsequent request to a protected endpoint, the token is included in the Authorization header as a Bearer token:

Authorization: Bearer <token>

2. Token Structure

The JWT payload contains the user ID and any other non-sensitive data (e.g., role). Example payload:

```json
{
  "id": "60d21b4667d0d8992e610c85",
  "iat": 1678900000,
  "exp": 1681492000
}
```

The token is signed with the server's secret, preventing tampering.

3. Protecting Routes (Authorisation)

· Authentication Middleware

A custom middleware function (protect) is applied to routes that require a logged-in user. It:

1. Extracts the token from the Authorization header.

2. Verifies the token using the secret key.

3. If valid, fetches the corresponding user from the database (excluding the password) and attaches it to the request object (req.user).

4. If the token is missing or invalid, it returns a 401 Unauthorized response.

· Role-Based Authorisation

For admin-only actions (e.g., creating or deleting books), an additional middleware (admin) checks if the authenticated user has the role 'admin'. If not, it returns 403 Forbidden.

4. Example Flow

1. User logs in and receives a token.

2. Frontend stores the token.

3. When the user attempts to view their profile (GET /api/auth/profile), the frontend includes the token in the request header.

4. The backend protect middleware validates the token and attaches the user to req.

5. The controller then returns the user's profile data.

5. Security Considerations

· Password Hashing: All passwords are hashed with bcrypt (salt rounds = 10) before storage.

· Token Expiration: Tokens expire after a set period (e.g., 30 days) to limit misuse. The client must handle token renewal (e.g., by re-logging in or using a refresh token – not implemented in this version).

· Environment Variables: The JWT secret is stored in .env and never hard-coded.

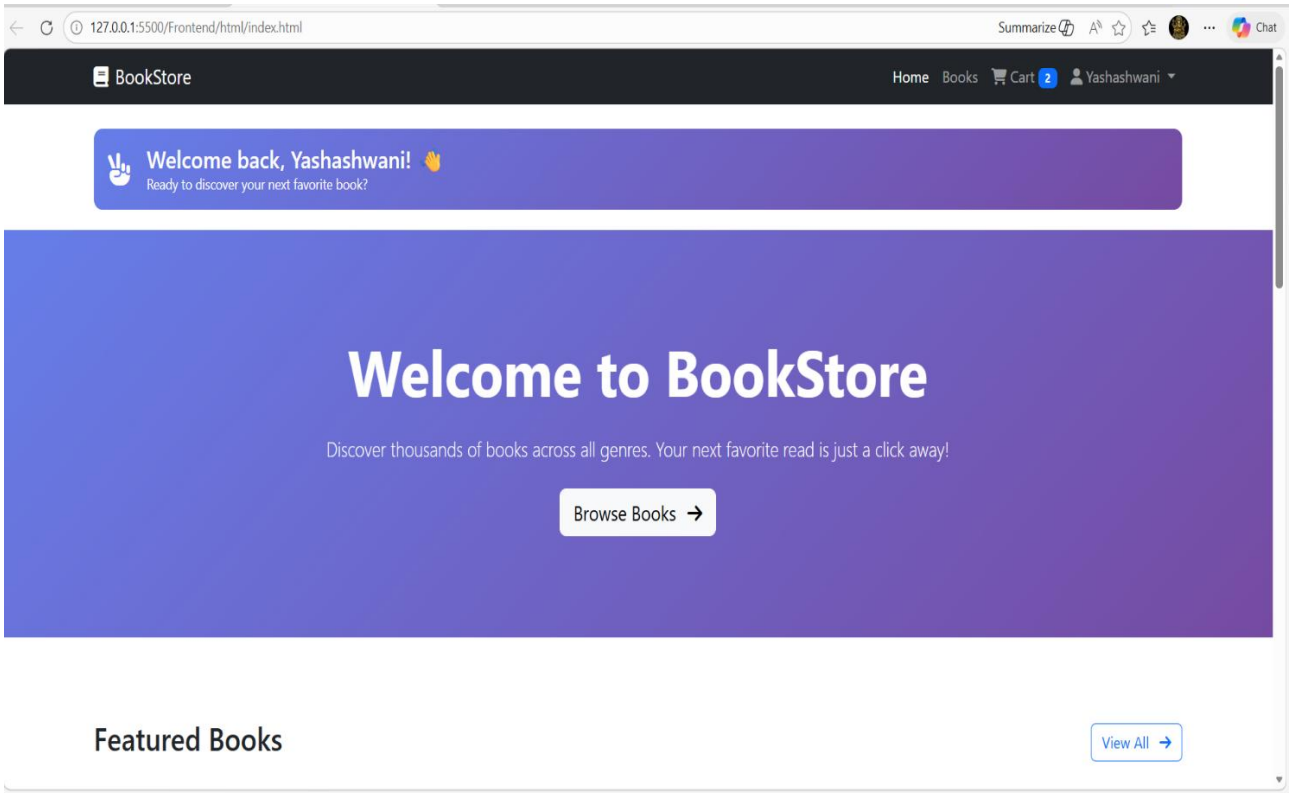
· HTTPS: In production, all communication should occur over HTTPS to prevent token interception.

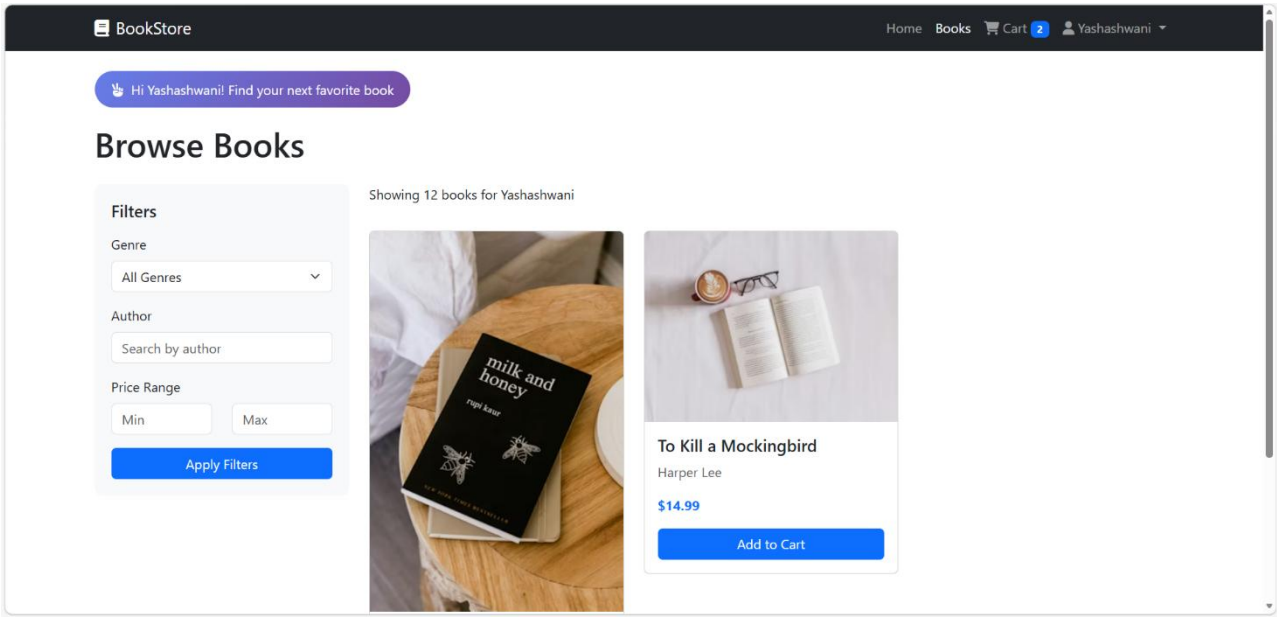
· XSS Protection: Storing tokens in localStorage can be vulnerable to XSS attacks. In a production app, consider using HTTP-only cookies for enhanced security, but for simplicity, this project uses localStorage with appropriate frontend sanitisation.

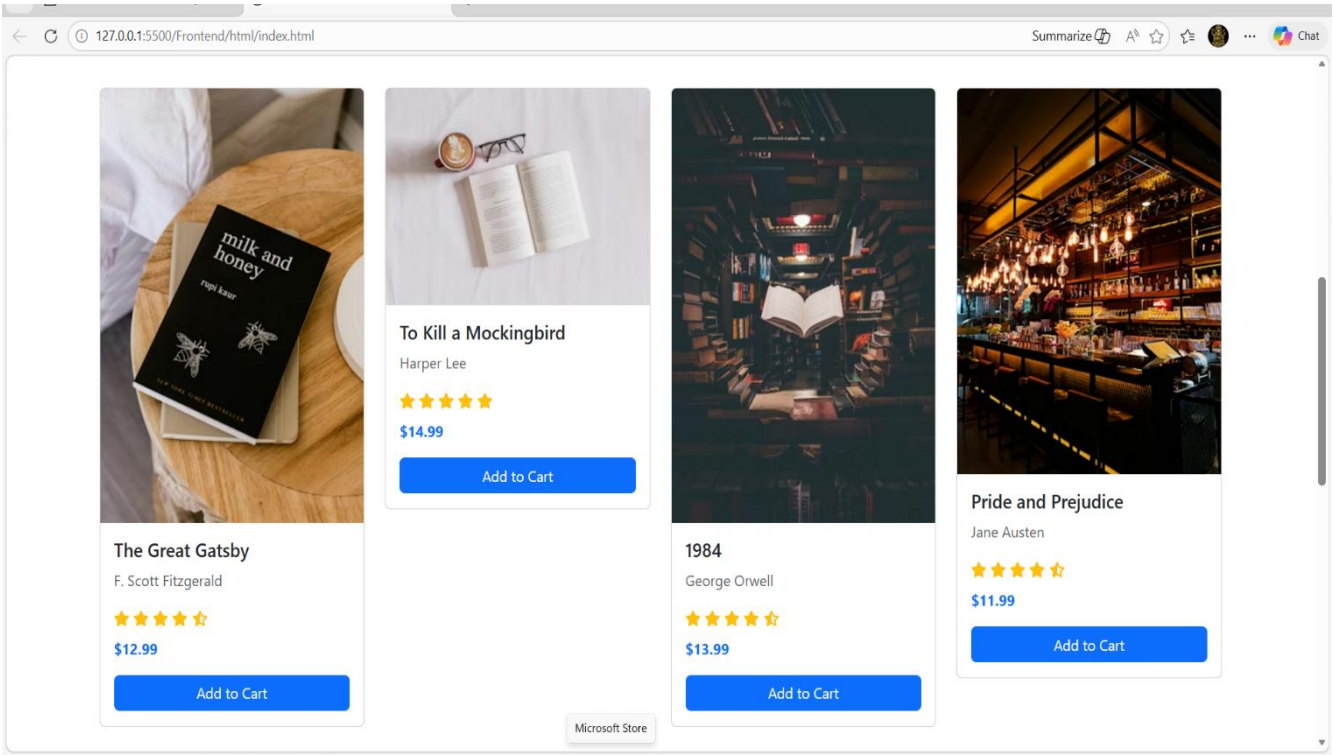
6. Summary

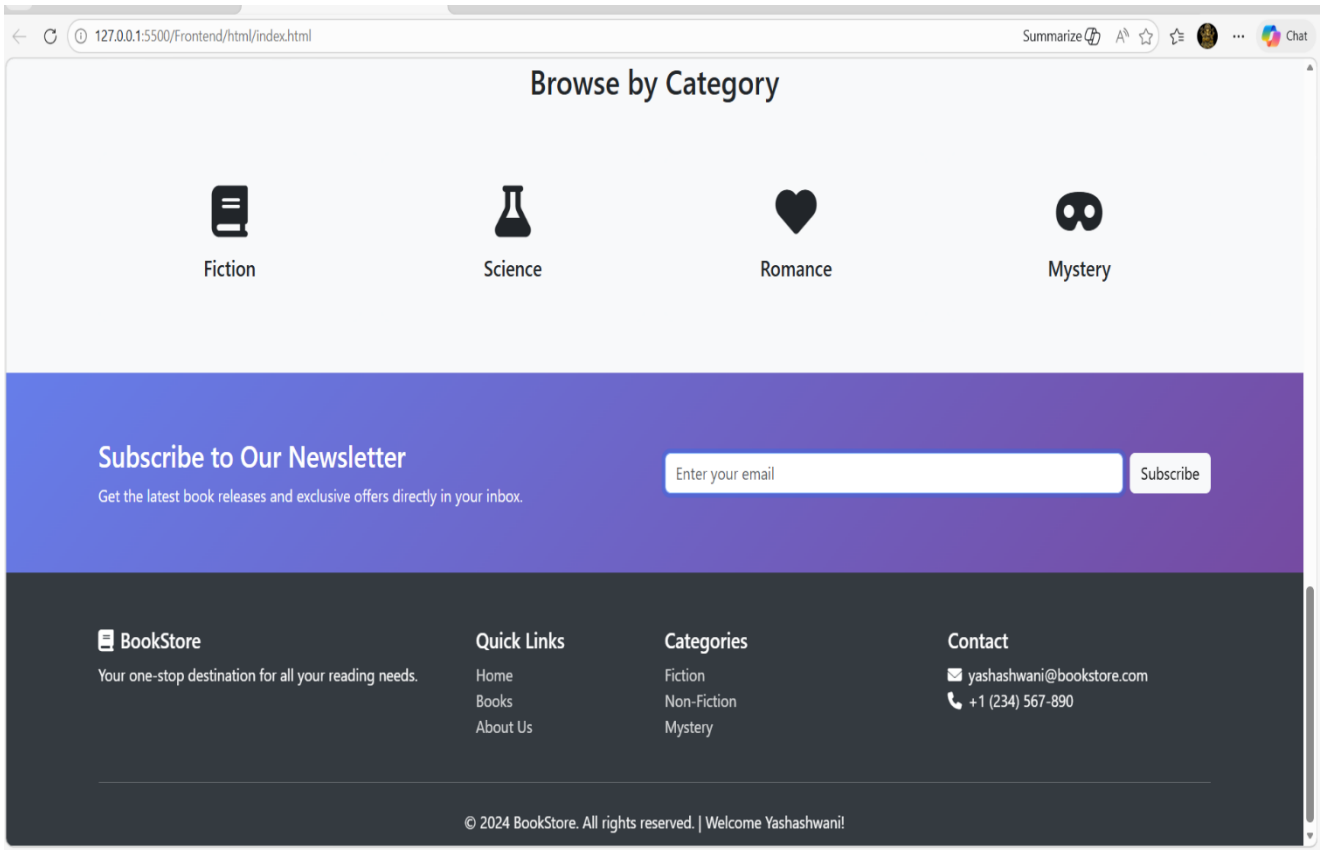
The combination of JWT and middleware provides a clean, scalable authentication and authorisation mechanism. It ensures that only authenticated users can access protected resources, and only administrators can perform privileged operations.

9 User Interface






127.0.0.1:5500/Frontend/html/index.html
Summarize
Chat
milk and honey
rupi kaur
The Great Gatsby
F. Scott Fitzgerald
$12.99
Add to Cart
To Kill a Mockingbird
Harper Lee
$14.99
Add to Cart
1984
George Orwell
$13.99
Add to Cart
Pride and Prejudice
Jane Austen
$11.99
Add to Cart
Microsoft Store


127.0.0.1:5500/Frontend/html/index.html
Summarize
Chat
Browse by Category
Fiction
Science
Romance
Mystery
Subscribe to Our Newsletter
Get the latest book releases and exclusive offers directly in your inbox.
Enter your email
Subscribe
BookStore
Your one-stop destination for all your reading needs.
Quick Links
Home
Books
About Us
Categories
Fiction
Non-Fiction
Mystery
Contact
yashashwani@bookstore.com
+1 (234) 567-890
© 2024 BookStore. All rights reserved. | Welcome Yashashwani!

10 Testing

Testing Strategy

To ensure the reliability, functionality, and performance of the Book Store Application, a multi-level testing strategy is employed. The approach encompasses unit testing, integration testing, and end-to-end (E2E) testing, covering both the frontend and backend components.

1. Testing Levels

➢ **Unit Testing**

· Backend: Individual functions (controllers, middleware, utility functions) are tested in isolation. Mocking of database calls (using jest.mock) and external dependencies ensures that each unit behaves correctly.

· Frontend: React components are tested in isolation using Jest and React Testing Library. Tests verify component rendering, user interactions, and state changes without relying on the actual backend.

➢ **Integration Testing**

· Backend: API endpoints are tested by sending real HTTP requests to the server while using a test database (e.g., an in-memory MongoDB instance). This validates the complete request-response cycle, including database interactions and middleware.

· Frontend: Integration tests check how multiple components work together (e.g., a page component that fetches data and renders a list). API calls are intercepted using Mock Service Worker (MSW) or similar to simulate backend responses.

➢ **End-to-End (E2E) Testing**

· Full user flows (e.g., user registration → login → browse books → add to cart → checkout) are tested using Cypress or Playwright. These tests run in a real browser environment, ensuring that the entire system works as expected from the user's perspective.

2. Backend Testing

· Framework: Jest (with supertest for HTTP assertions).

· Test Database: A separate test database (or an in-memory MongoDB server using mongodb-memory-server) is used to avoid polluting the development database.

· **What is tested:**

· Authentication: successful/failed login, token generation, protected routes.

· Book endpoints: CRUD operations, filtering, search, and review submission.

· Order endpoints: order creation, retrieval, and cancellation, including inventory updates.

· Mocking: External services (e.g., email sending) are mocked to avoid side effects.

3. Frontend Testing

· Framework: Jest combined with React Testing Library.

· What is tested:

· Component rendering: presence of key elements, conditional rendering (e.g., login vs. logout).

· User interactions: button clicks, form submissions, cart updates.

· Context behaviour: authentication and cart state changes.

· API integration: using MSW to simulate responses from the backend.

· Snapshot testing: Used sparingly for stable components to detect unexpected UI changes.

4. End-to-End Testing

· Tool: Cypress (or Playwright).

· **What is tested:**

· Complete user journeys:

· Guest browsing and adding items to cart.

· User registration and login.

· Placing an order and viewing order history.

· Admin actions (if applicable).

· Cross-browser compatibility (simulated via Cypress).

· Responsive design (testing on different viewport sizes).

5. Tools Used

Tool Purpose

Jest Test runner and assertion library for both frontend and backend.

Supertest Testing HTTP endpoints (used with Jest).

React Testing Library Rendering React components and simulating user interactions.

Mock Service Worker (MSW) Intercepting network requests in frontend tests to mock API responses.

MongoDB Memory Server Providing an isolated in-memory MongoDB instance for backend tests.

Cypress End-to-end testing in the browser.

ESLint Static code analysis to catch potential errors and enforce style.

Prettier Code formatting to maintain consistency.

6. Continuous Integration (CI)

· GitHub Actions (or similar) is set up to automatically run tests on every push and pull request.

· The CI pipeline:

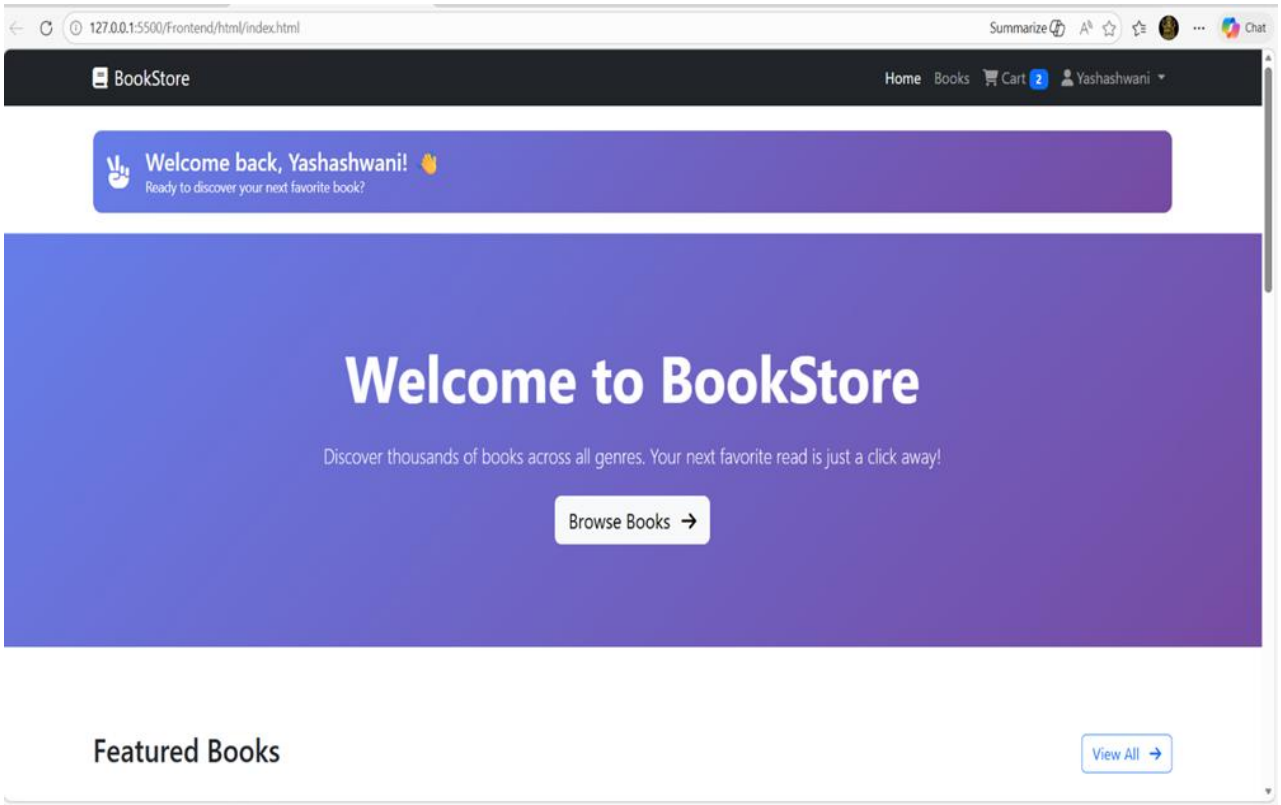
1. Installs dependencies.
2. Runs linters (ESLint, Prettier check).
3. Executes backend and frontend unit/integration tests.
4. Builds the frontend application.
5. (Optional) Runs E2E tests against a deployed preview.

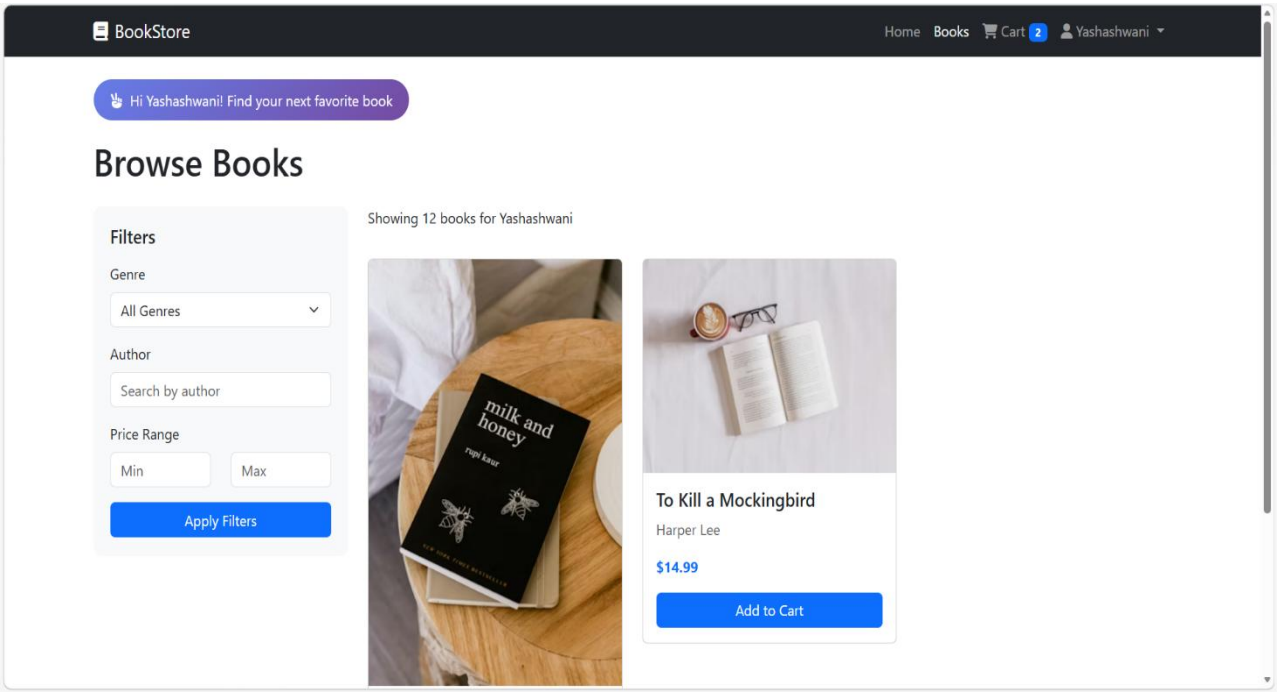
7. Manual Testing

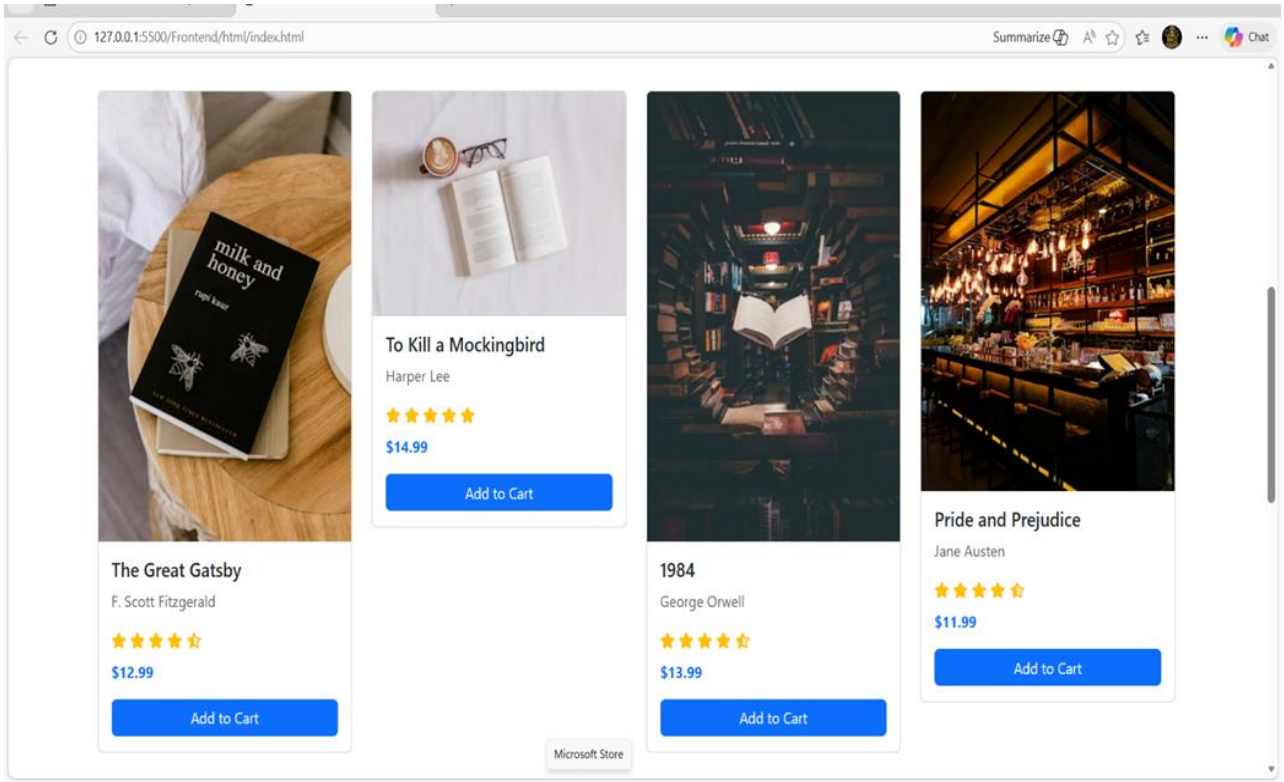
In addition to automated tests, exploratory manual testing is performed to catch usability issues and edge cases that automated scripts may miss. This includes testing on different browsers and devices to ensure a consistent experience.

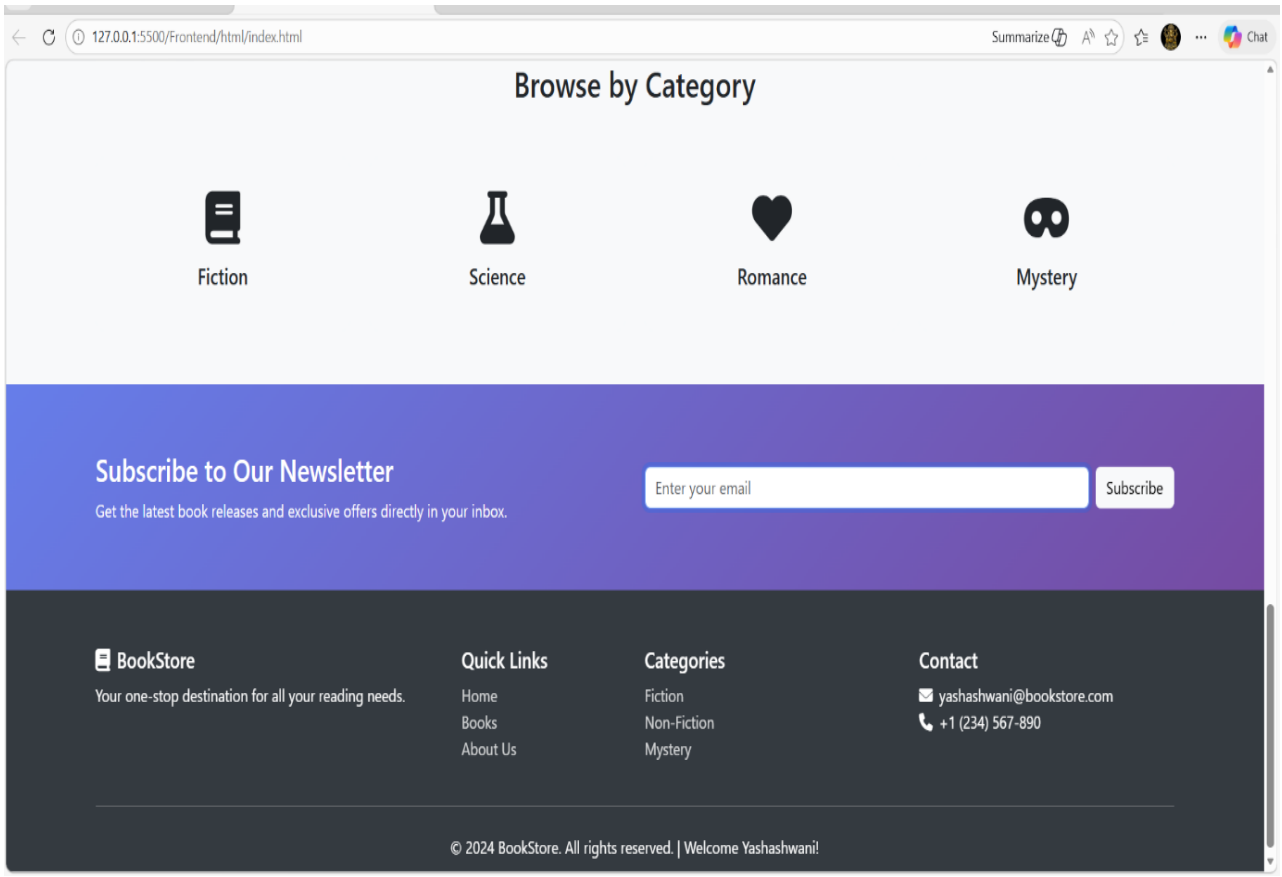
This comprehensive testing strategy ensures that the Book Store Application remains robust, maintainable, and user-friendly as it evolves.

11 Screenshots or Demo






127.0.0.1:5500/Frontend/html/index.html
Summarize
Chat
milk and honey
rupi kaur
To Kill a Mockingbird
Harper Lee
$14.99
Add to Cart
The Great Gatsby
F. Scott Fitzgerald
$12.99
Add to Cart
1984
George Orwell
$13.99
Add to Cart
Pride and Prejudice
Jane Austen
$11.99
Add to Cart
Microsoft Store


127.0.0.1:5500/Frontend/html/index.html
Summarize
Chat
Browse by Category
Fiction
Science
Romance
Mystery
Subscribe to Our Newsletter
Get the latest book releases and exclusive offers directly in your inbox.
Enter your email
Subscribe
BookStore
Your one-stop destination for all your reading needs.
Quick Links
Home
Books
About Us
Categories
Fiction
Non-Fiction
Mystery
Contact
yashashwani@bookstore.com
+1 (234) 567-890
© 2024 BookStore. All rights reserved. | Welcome Yashashwani!

12 Known Issues

Known Issues & Limitations

The following issues and limitations have been identified in the current version of the Book Store Application. Users and developers should be aware of them when using or extending the project.

Authentication & Security

· JWT stored in localStorage – This makes the token vulnerable to XSS attacks. A more secure alternative would be HTTP-only cookies.

· No token refresh mechanism – Tokens expire after a fixed period (30 days) and cannot be renewed without re-logging in.

· Password reset not implemented – Users cannot recover their accounts if they forget their password.

· No email verification – New accounts are created without verifying the email address, which could lead to fake registrations.

Payment Integration

· No real payment gateway – Payment is simulated; no actual transactions are processed. Integration with services like Stripe or PayPal is planned but not yet implemented.

Search & Filtering

· Basic search – Uses MongoDB's $text search, which does not support fuzzy matching or typo tolerance.

· Single-genre filter – Only one genre can be selected at a time; multi-select is not available.

· Price filter limitations – No currency conversion; international users may see prices in USD only.

Performance

· No caching – Frequently requested data (e.g., book lists, genres) is not cached, causing repeated database queries.

· Image optimisation – Book covers are loaded without optimisation (e.g., resizing, lazy loading), which may impact page load speed.

· Slow pagination on large datasets – The total count query can become slow as the number of books grows.

Frontend

· Cart persistence – Relies on localStorage; clearing browser data will erase the cart.

· No guest checkout – Users must register before placing an order.

· Basic form validation – Frontend validation is minimal; improved real-time feedback is needed.

· Accessibility (a11y) shortcomings – ARIA labels and keyboard navigation could be enhanced.

Backend

· No rate limiting – The API is susceptible to brute-force attacks on login endpoints.

· Minimal logging – Only console logs are used; a structured logging system (e.g., Winston) is absent.

· Missing environment variable validation – If required .env variables are not set, the application may fail silently.

Admin Features

· Limited admin interface – No dedicated dashboard for managing orders or users (though API endpoints exist).

· No low-stock alerts – Inventory management lacks notifications when stock levels are low.

General

· English-only UI – No multi-language support.

· Basic SEO – React Helmet provides meta tags, but sitemap generation and advanced SEO are missing.

· Generic error messages – Some errors are displayed as non-specific messages, reducing user-friendliness.

· Test coverage is not exhaustive – Some edge cases and error scenarios lack automated tests.

These issues do not prevent the application from functioning as a demonstration or learning tool, but they should be addressed before deploying to a production environment. Contributions and pull requests are welcome to improve any of these areas.

13 Future Enhancements

Future Enhancements

The following enhancements are planned for future releases to extend the functionality, improve user experience, and address current limitations.

1. Real Payment Gateway Integration

· Integrate popular payment gateways like Stripe, PayPal, or Razorpay to enable actual online transactions.

· Support multiple payment methods (credit/debit cards, digital wallets, net banking).

2. User Authentication Improvements

· Social login – Allow users to sign in using Google, Facebook, or GitHub accounts.

· Two-factor authentication (2FA) – Add an extra layer of security.

· Password reset functionality – Implement a secure "forgot password" flow via email.

· Email verification – Send a verification link upon registration to confirm email addresses.

3. Enhanced Search & Filtering

· Implement elasticsearch or Algolia for fast, typo-tolerant full-text search.

· Add multi-select filters (e.g., multiple genres, authors, price ranges).

· Introduce advanced sorting (by popularity, bestsellers, user ratings).

4. User Experience & Features

· Wishlist – Allow users to save books for later purchase.

· Recently viewed books – Track and display books the user has viewed.

· Personalised recommendations – Suggest books based on purchase history and browsing behaviour.

· Guest checkout – Enable ordering without mandatory registration.

· PDF/e-book previews – Allow users to preview a few pages before buying.

5. Admin Dashboard Enhancements

· Build a dedicated admin panel with analytics (sales reports, top-selling books, user statistics).

· Inventory management – Set low-stock alerts and automated reorder suggestions.

· Order management – Bulk update order status, print invoices, and generate shipping labels.

6. Performance Optimisation

· Implement caching (Redis or in-memory caching) for frequently accessed data like book listings and genres.

· Image optimisation – Use CDN, lazy loading, and responsive images to improve load times.

· Database indexing – Review and add indexes to speed up complex queries.

7. Security Enhancements

· Move JWT storage to HTTP-only cookies to mitigate XSS risks.

· Add rate limiting to prevent brute-force attacks on authentication endpoints.

· Implement CSRF protection for state-changing operations.

· Use helmet.js to set secure HTTP headers.

8. Mobile Application

· Develop a React Native or Flutter mobile app to provide a native experience on iOS and Android.

· Share code with the existing React web app where possible.

9. Internationalisation (i18n)

· Add support for multiple languages (e.g., Spanish, French, Hindi) using react-i18next.

· Allow users to switch language and see translated content.

10. Progressive Web App (PWA)

· Convert the frontend into a PWA with offline support, push notifications, and home screen installation.

· Use service workers to cache assets and enable offline browsing of previously viewed books.

11. API Versioning & Documentation

· Introduce API versioning (e.g., /api/v1/books) to support future changes without breaking existing clients.

· Generate interactive API documentation using Swagger/OpenAPI.

12. Continuous Integration & Deployment

· Set up a CI/CD pipeline (e.g., GitHub Actions, GitLab CI) to automatically run tests and deploy to staging/production environments.

· Implement automated end-to-end testing as part of the pipeline.

These enhancements would transform the Book Store Application into a production-ready, feature-rich platform capable of serving a large user base and competing with established e-commerce solutions.